

Day 1 - C++ Foundations

You will read a game codebase far more often than you write it. Before we touch memory, classes, or pathfinding, we settle two things: how the code *looks*, and how we stop data from being changed by accident. `const`-correctness and a clean split between headers and sources are what let other people extend your code later, and they are the first thing a reviewer sees.

Learning objectives

By the end of today you can:

- [ ] Apply `const` to variables, parameters, and return types and explain what each guards.
- [ ] Pass objects by const reference to avoid copies without allowing mutation.
- [ ] Write const member functions and explain what `this` becomes inside them.
- [ ] Split a class cleanly into header (`.hpp`) and source (`.cpp`) guarded by `#pragma once`.
- [ ] Follow a consistent naming convention across a small codebase.

Concepts

`const` basics

`const` promises the compiler a value won't change through that name, and the compiler holds you to it. "Make it `const` until you can't" is the default: it documents intent and catches accidental writes at compile time.

```
const int MAX_HP = 100;    // value never changes
int hp = MAX_HP;           // fine: a fresh, mutable copy

const int* p = &hp;        // low-level const: can't write *p ...
p = &MAX_HP;               //    ... but can repoint p
int* const q = &hp;        // top-level const: can write *q,
can't repoint q
```

❌ **Common Mistake:** reaching for `const` only on "important" values.

✅ **Good Practice:** default to `const` everywhere and drop it only where you genuinely need to mutate.

const references

`const T&` passes a read-only view: nothing is copied, and the callee cannot mutate the argument. Use it for any object you only read. For small trivially-copyable types (`int`, `float`, a 2-float `Vec2`) a plain by-value copy costs the same and reads more clearly.

```
float length(const Vec2& v);   // & avoids copying Vec2; const keeps caller's v safe
void  tick(int deltaMs);       // int fits in a register, so copying it is free
```

❌ **Common Mistake:** `const&` on a tiny type, or a *non*-const `&` on a parameter you only read. The non-const reference quietly forbids temporaries and literals, so `length(Vec2(1, 2))` stops compiling.

✅ **Good Practice:** pass small trivially-copyable types by value, and take everything you only read as `const T&`.

const member functions

A trailing `const` makes the method promise not to modify the object. Inside it, `this` is a pointer-to-const, so the members are read-only. Only const methods can be called on a `const` object, which is why accessors *must* be const to be usable at all.

```
class Vec2 {
public:
    Vec2(float x, float y) : m_x(x), m_y(y) {}
    float x() const{ return m_x; }   // reads only -> const
    float y() const{ return m_y; }
    void  setX(float v){ m_x = v; }  // mutates -> NOT const
private:
    float m_x;
    float m_y;
};

const Vec2 origin(0.0f, 0.0f);   // a const object needs a constructor to initialise it
origin.x();       // ok
origin.setX(1);   // compile error: setX is not const
```

❌ **Common Mistake:** forgetting `const` on getters. Every accessor call on a `const` object is then a compile error, which leaves the object unusable.

✅ **Good Practice:** mark every method that doesn't mutate `const`, so a `const Vec2` stays fully readable.

Naming conventions

One scheme, applied everywhere. **The convention for this gauntlet:**

Kind	Style	Example
Types (class/struct/enum)	`PascalCase`	`EnemySpawner`, `Vec2`
Methods / free functions	`camelCase`	`activeCount()`, `length()`
Member variables	`m_camelCase`	`m_activeCount`, `m_spawnRate`
Locals / parameters	`camelCase`	`deltaMs`, `spawnRate`
Constants / macros	`UPPER_SNAKE`	`MAX_ENEMIES`
Files	match the class	`EnemySpawner.hpp` / `.cpp`

The `m_` prefix makes "is this a member or a local?" answerable at a glance and avoids `this->` clutter in constructors. Pick meaningful names over abbreviations.

Header / source structure

Declarations go in the `.hpp`, definitions in the `.cpp`. Headers stay light so they compile fast and don't drag every includer into a rebuild. Use a forward declaration instead of an `#include` when you only need a pointer or reference to a type.

```
// Enemy.hpp
#pragma once

class World;                 // forward decl: we only hold a
World& here

class Enemy {
public:
    Enemy(World& world);
    void update(int deltaMs);   // declared here ...
private:
    World& m_world;
};
```

```
// Enemy.cpp
#include "Enemy.hpp"
```

```
#include "World.hpp"          // full definition needed only in
the .cpp

Enemy::Enemy(World& world) : m_world(world) {}
void Enemy::update(int deltaMs){ /* ... define here */ }
```

❌ **Common Mistake:** putting a non-inline function *definition* in a header, then including it from two `.cpp` files. That breaks the one-definition rule, and the linker rejects it as a duplicate symbol.

✅ **Good Practice:** declare in the `.hpp`, define in the `.cpp`. When a definition must live in the header, mark it `inline` or write it inside the class body.

Include guards (`#pragma once`)

A header may be reached twice in one translation unit (e.g. `main.cpp` includes both `Enemy.hpp` and `World.hpp`, and both pull in `Vec2.hpp`). The compiler would then see `class Vec2` declared twice and reject the redefinition. `#pragma once` makes the second inclusion a no-op: the compiler remembers each file it has already read and skips it. It goes on the first line of every `.hpp` you write.

```
// Vec2.hpp
#pragma once                // first line, before any include
or declaration

// ... declarations ...
```

It identifies the *file*, not the header's name. Two copies of the same header at different paths are two different files, and both get included. Keep one copy of each header in your include path and this never bites you.

❌ **Common Mistake:** leaving it off a new header. The file compiles fine while only one `.cpp` includes it. The day a second includer appears, the build breaks with a redefinition error that points at the *including* file rather than the unguarded header.

✅ **Good Practice:** type `#pragma once` as the first line of a header the moment you create the file, before you write a single declaration.

Exercises

ex00 - A const-correct value type

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*Implement a tiny value type while enforcing const-correct APIs (const accessors, const methods, no mutation through const).*
📂	**Turn in directory**	`ex00/`
❗	**Turn in files**	**Vec2.hpp, Vec2.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*Write a header/source pair, compile with warnings-as-errors, read compiler diagnostics for const violations.*
🔍	**Search keywords**	*const member function, this pointer const, top-level vs low-level const, pass by const reference, pragma once*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<cmath>`

- **Goal:** A small `Vec2` (two `float`s) with:
  - a constructor taking `x` and `y` (a `const Vec2` can't be default-initialised without one),
  - const accessors `x()` / `y()`,
  - a `length() const` returning the magnitude,
  - a `plus(const Vec2&) const` returning a new `Vec2` (does not mutate either operand),
  - mutators `setX` / `setY`,
  - `#pragma once` at the top of the header, and no member mutated by a `const` method. In `main.cpp`, construct a `const Vec2`, read it, add two vectors, and print results.
- **Done when:** A `const Vec2` compiles when read but the compiler *rejects* a write to it; the build is warning-clean under the flags above.

Output example:

```
$ ./vec2
a = (3, 4), length 5
b = (1, 2), length 2.23607
a + b = (4, 6)              # from plus(), neither operand
changed
```

```
b now (10, 2)                # from setX(), the one allowed
mutation
```

`a + b` comes from a `const` method that returns a fresh `Vec2`, so you compute a result without mutating either operand.

ex01 - Header hygiene

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Practice correct separation of declarations vs definitions, and put includes only where they belong.*
📁	**Turn in directory**	`ex01/`
❗	**Turn in files**	**Timer.hpp, Timer.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*Guarding a header with `#pragma once`, knowing what belongs in .hpp vs .cpp, compiling multiple translation units.*
🔍	**Search keywords**	*header vs source, pragma once, forward declaration vs include, one definition rule, multiple definition linker error*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`

- **Goal:** Start from this single-file blob and split it correctly: declarations in the header (guarded by `#pragma once`), definitions in the source, and each include where it actually belongs. Nothing that belongs in the `.cpp` should leak into the `.hpp`.

```
// starter.cpp: split this into Timer.hpp / Timer.cpp /
main.cpp
#include <iostream>
class Timer {
public:
    Timer(int startMs) : m_elapsedMs(startMs) {}
    void advance(int deltaMs){ m_elapsedMs += deltaMs; }
    int elapsedMs() const{ return m_elapsedMs; }
private:
    int m_elapsedMs;
};
int main(){
    Timer t(0);
    t.advance(16);
    std::cout << t.elapsedMs() << '\n';
}
```

- **Done when:** `Timer.hpp` can be `#include`d twice in one translation unit without error, and only the `.cpp` files carry function bodies.

Output example:

```
$ ./timer
16
```

Behaviour is unchanged from the blob: `advance(16)` on a timer that started at 0 prints `16`. The split is purely structural, so if the output changed, the refactor broke something.

ex02 - Naming pass

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*Apply the gauntlet naming conventions consistently without changing behaviour.*
📁	**Turn in directory**	`ex02/`
❗	**Turn in files**	**EnemySpawner.hpp, EnemySpawner.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*Refactoring carefully, updating identifiers consistently across header/source, validating behaviour still matches.*
🔍	**Search keywords**	*C++ naming conventions m_ prefix, PascalCase vs camelCase, const getter, refactor rename symbol*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`

- **Goal:** Take this "bad style" code and bring it fully in line with the gauntlet naming convention (see **Naming conventions** above). Behaviour must be identical, since this is a pure rename and restructure pass. Split it into `EnemySpawner.hpp` / `.cpp`, and drive it from the `main.cpp` shown below, which already uses the target names `spawn()` and `spawnedCount()`.

```
// bad_style.cpp: rename everything to the convention, keep
behaviour identical
class enemy_spawner {
  public:
    enemy_spawner(int MaxCount) { COUNT = MaxCount; spawned
= 0; }
    int Spawn(){ if (spawned < COUNT) spawned++; return
spawned; }
    int GetSpawned(){ return spawned; }   // should this be
const?
  private:
    int COUNT;
    int spawned;
};
```

```
// main.cpp: spawn past the cap, then read the count back
#include <iostream>
#include "EnemySpawner.hpp"

int main(){
    EnemySpawner spawner(2);
    std::cout << "spawned " << spawner.spawn() << '\n';   //
1
    std::cout << "spawned " << spawner.spawn() << '\n';   //
2
    std::cout << "spawned " << spawner.spawn() << '\n';   //
2 (capped)
    std::cout << "total   " << spawner.spawnedCount() <<
'\n';
}
```

- **Done when:** `enemy_spawner` follows the `PascalCase` rule for types, `Spawn` and `GetSpawned` follow the `camelCase` rule for methods, and `COUNT` and `spawned` carry the `m_` prefix. Decide for yourself which members and methods stay mutable and which become `const`. You will be asked to justify that call.

Output example:

```
$ ./spawner
spawned 1
spawned 2
```

```
spawned 2
total   2
```

Only the names changed. The cap still holds at 2, so the third `spawn()` returns 2 and `spawnedCount()` reads 2.

Constraints (every exercise)

- Compiles with `c++ -Wall -Wextra -Werror -std=c++17`.
- One class per header/source pair.
- No function definitions in headers except genuinely inline one-liners.
- Each `ex*/` builds with its own `Makefile` (`all` / `clean` / `fclean` / `re`).

How you will be evaluated

An evaluation will be automatically scheduled with a peer and start at 6pm. Submit all the work you have done so far even if its not 100% complete.

Turning in your work

1. Create a new repository on GitHub named `gauntlet-day-01`, and keep it **public**.
2. Commit today's work there, one directory per exercise (`ex00/`, `ex01/`, ...) exactly as each turn-in table says, `Makefile` included.
3. Push all the work before the evaluation.
4. When the evaluation starts, give your evaluator the link to your repository.

AI-written code is an immediate disqualification. Look things up and ask an AI what something does, that is a tool. Pasting in code you did not write is not.